

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

2.1 ภาษา PHP

ในปัจจุบันเว็บไซต์ (Web Site) ต่าง ๆ ได้มีการพัฒนาในด้านต่างๆ อย่างรวดเร็ว อาทิเช่น เรื่องของความสวยงาม และแปลกใหม่ , การบริการข่าวสารข้อมูลที่ทันสมัย , เป็นสื่อกลางในการติดต่อ และสิ่งหนึ่งที่กำลังได้รับความนิยมเป็นอย่างมากซึ่ง ได้ว่าเป็นการปฏิวัติรูปแบบการ ขายของ ก็คือ E-Commerce ซึ่งเจ้าของสินค้าต่างๆ ไม่จำเป็นต้องมีร้านค้าจริง และไม่จำเป็นต้องจ้างคนขายของอีกต่อไปร้านค้า และตัวสินค้านั้น จะไปปรากฏอยู่บนเว็บไซต์ และการซื้อขายก็เกิดขึ้นบนโลกของ Internet แล้ว PHP ช่วยเราให้เป็นการพัฒนาเว็บไซต์ และความสามารถที่โดดเด่นอีกประการหนึ่งของ PHP นั้น คือ Database-enabled web page ทำให้เอกสารของ HTML สามารถที่จะเชื่อมต่อกับระบบฐานข้อมูล (Database) ได้อย่างมีประสิทธิภาพและรวดเร็ว จึงทำให้ ความต้องการในเรื่องการจัดรายการสินค้า และรับรายการสั่งของตลอดจนการจัดเก็บ ข้อมูลต่างๆ ที่สำคัญผ่านทางอินเทอร์เน็ต (Internet) เป็นไปได้อย่างง่ายดาย PHP เป็นภาษาจาวาสคริปต์ Scripting Language คำสั่งต่างๆ จะเก็บอยู่ในไฟล์ที่เรียกว่า สคริปต์ (Script) และเวลาใช้งานต้องอาศัยตัวแปลชุดคำสั่ง ตัวอย่างของภาษาสคริปต์ก็เช่น จาวาสคริปต์ (Java Script), Perl เป็นต้น ลักษณะของ PHP ที่แตกต่างจากภาษาสคริปต์แบบอื่นๆ คือ PHP ได้รับการพัฒนาและออกแบบมาเพื่อใช้ในการสร้างเอกสารแบบ HTML โดยสามารถสอดแทรกหรือแก้ไขเนื้อหาได้โดยอัตโนมัติ ดังนั้นจึงกล่าวว่า PHP เป็นภาษาที่เรียกว่า Server-Side

เนื่องจากว่า PHP ไม่ได้เป็นส่วนหนึ่งของตัวเว็บเซิร์ฟเวอร์ดังนั้นถ้าจะใช้ PHP ก็จะต้องดูก่อนว่าเว็บเซิร์ฟเวอร์นั้นสามารถใช้สคริปต์ PHP ได้หรือไม่ ยกตัวอย่างเช่น PHP สามารถใช้ได้กับ Apache Web Server และ Personal Web Server (PWP) สำหรับระบบปฏิบัติการ Windows 95/98/NT ในกรณีของ Apache เราสามารถใช้ PHP ได้สองรูปแบบคือ ในลักษณะของ CGI และ Apache Module ความแตกต่างอยู่ตรงที่ว่า ถ้าใช้ PHP เป็นแบบโมดูล PHP จะเป็นส่วนหนึ่งของ Apache หรือเป็นส่วนขยายในการทำงานนั่นเอง ซึ่งจะทำงานได้เร็วกว่าแบบที่เป็น CGI เพราะว่า ถ้าเป็น CGI แล้ว ตัวแปลชุดคำสั่งของ PHP ถือว่าเป็นแค่โปรแกรมภายนอก ซึ่ง Apache จะต้องเรียกขึ้นมาทำงานทุกครั้ง ที่ต้องการใช้ PHP ดังนั้น ถ้ามองในเรื่องของประสิทธิภาพในการทำงาน การใช้ PHP แบบที่เป็นโมดูลหนึ่งของ Apache จะทำงานได้มีประสิทธิภาพมากกว่า

ลักษณะเด่นของ PHP

- ใช้งานได้ฟรี เป็น Open Source
- เป็น โปรแกรม Run ฝั่ง Sever ดังนั้น จัดความสามารถไม่จำกัด
- PHP สามารถทำงานได้กับระบบปฏิบัติการที่หลากหลาย Windows, UNIX และ Linux
- เรียนรู้ง่าย เนื่องจาก PHP ผังเข้าไปใน HTML และใช้โครงสร้างและไวยากรณ์ภาษาง่ายๆ
- เร็ว และมีประสิทธิภาพ
- ใช้ร่วมกับ XML ได้ทันที
- ใช้กับระบบเพิ่มข้อมูลได้
- ใช้กับข้อมูลตัวอักษรได้อย่างมีประสิทธิภาพ
- ใช้กับ โครงสร้างข้อมูลใช้ได้แบบ Scalar , Array , Associative array
- ใช้กับการประมวลผลภาพได้

2.2 ระบบฐานข้อมูล MySQL

การสร้างเว็บเพจที่เรียบง่าย และไม่มีการเปลี่ยนแปลงบ่อย ๆ นั้น การใช้ภาษา HTML เพียงอย่างเดียวก็อาจจะเพียงพอแล้ว แต่ในปัจจุบันความต้องการรับรู้ข่าวสารต่าง ๆ ให้รวดเร็ว และทันต่อเหตุการณ์ เป็นแรงกดดันให้ผู้ที่เคยใช้เพียงภาษา HTML จำเป็นต้องนำเทคโนโลยีอื่นๆ เข้ามาช่วยให้สามารถปรับปรุงข้อมูลได้ทันที และลดความผิดพลาดที่จะเกิดขึ้นจากการที่ต้องปฏิบัติงานกับข้อมูลจำนวนมาก และสิ่งหนึ่งที่ถูกนำมาใช้เพื่อบรรเทาปัญหาเหล่านี้ คือ ระบบฐานข้อมูลบนเว็บ (Web Database)

เมื่อมีความจำเป็นต้องนำระบบจัดการฐานข้อมูลมาใช้ในเว็บ ไซต์ที่เขียนขึ้นด้วยภาษา PHP ระบบจัดการฐานข้อมูลที่มีจะถูกเลือกใช้เป็นลำดับต้น ๆ คือ MySQL ด้วยคุณสมบัติเด่นหลายประการ ทำให้ผู้ใช้ PHP ส่วนใหญ่เลือกใช้ MySQL เป็นระบบฐานข้อมูลสำหรับเว็บ ไซต์ และเว็บแอปพลิเคชัน

MySQL (อ่านว่า มาย-เอส-คิว-แอล “My Ess Que Ell ไม่ใช่ My Sequel”) เป็นระบบฐานข้อมูลที่ถูกพัฒนาโดยบริษัท MySQL AB ประเทศสวีเดน โดยผู้ก่อตั้งเป็นชาวสวีเดนสองคน คือ David Axmark และ Allan Larsson และชาวฟินแลนด์อีกคนหนึ่งคือ Michael “Monty” Widemius มีวัตถุประสงค์ให้ MySQL เป็นซอฟต์แวร์ฟรี ที่เปิดเผยซอร์สโค้ด GNU General Public License (GPL)

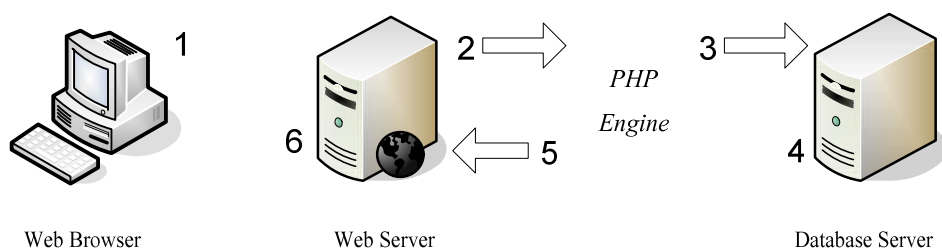
สถาปัตยกรรมเว็บคาต้าเบส

การทำงานของเว็บเซิร์ฟเวอร์โดยพื้นฐานจะเป็นการนำเว็บเพจที่ถูกเก็บอยู่ที่เซิร์ฟเวอร์ส่งผ่านไปให้แก่เว็บเบราว์เซอร์ในเครื่องที่ทำการร้องขอมา โดยข้อมูลส่วนใหญ่ในเว็บเพจจะอยู่ในรูปแบบของแท็กภาษา HTML และสคริปต์ที่ทำงานทางฝั่งไคลเอ็นต์ เช่น จาวาสคริปต์



รูปที่ 2-1 การทำงานของเว็บเซิร์ฟเวอร์

แต่สำหรับการนำระบบฐานข้อมูล เข้ามาใช้ร่วมกับเว็บเพจนั้น เมื่อเว็บเบราว์เซอร์ร้องขอมาที่เว็บเซิร์ฟเวอร์แล้ว PHP จะทำหน้าที่เป็นตัวกลางในการดึงข้อมูลจากฐานข้อมูล มาจัดทำเป็นสคริปต์ HTML ในรูปแบบที่เว็บเบราว์เซอร์สามารถเข้าใจได้ โดยมีขั้นตอน ดังรูป



รูปที่ 2-2 กระบวนการของเว็บแอปพลิเคชัน

จากรูป อธิบายกระบวนการของเว็บแอปพลิเคชันที่มีการติดต่อกับฐานข้อมูลได้ดังนี้

1. Web Browser ทำการร้องขอเว็บเพจด้วยโปรโตคอล HTTP ไปยัง Web Server
2. เมื่อ Web Server ได้รับการร้องขอ จะทำการเรียกไฟล์ที่ถูกร้องขอแล้วส่งต่อไปให้กับ PHP Engine เพื่อทำการประมวลผล
3. ในกรณีที่สคริปต์มีคำสั่งทำการติดต่อฐานข้อมูล และมีการทำคิวรี (Query) เพื่ออ่านหรือประมวลผลฐานข้อมูล PHP Engine ก็จะสามารถติดต่อและส่งคิวรีไปยังเว็บคาต้าเบสเซิร์ฟเวอร์ (Database Server)
4. Database Server จะส่งผลลัพธ์ของคิวรีกลับไปให้ PHP Engine

5. หลังจาก PHP Engine นำข้อมูลที่ได้รับจาก Database Server มาประมวลผลแล้ว จะทำการสร้างผลลัพธ์ในรูปแบบ HTML แล้วส่งให้แก่เซิร์ฟเวอร์

6. Server จะส่งผลลัพธ์ในรูปแบบ HTML กลับ ไปยังบราวเซอร์เพื่อแสดงผล

แม้ว่า MySQL ไม่ใช่ระบบจัดการฐานข้อมูลที่ดีที่สุด แต่ด้วยจุดเด่นหลายประการ ทำให้ MySQL ก้าวขึ้นมาเป็นระบบจัดการฐานข้อมูลที่มีผู้ใช้มากที่สุดในกลุ่ม Open Source Database

จุดเด่นของ MySQL

- ฟรี เป็น Open Source สามารถใช้งานได้โดยไม่ต้องเสียค่าลิขสิทธิ์
- มีความเร็วในการทำงานสูง
- ทำงานได้กับระบบปฏิบัติการที่หลากหลาย ไม่ว่าจะเป็น Windows , Linux และ Unix
- มีผู้ใช้เป็นจำนวนมาก ทำให้มีการพัฒนา และออกเวอร์ชันใหม่ ๆ อย่างสม่ำเสมอ
- ติดตั้ง และใช้งานง่าย มีคู่มือให้ดาวน์โหลดได้

ระบบจัดการฐานข้อมูล MySQL เหมาะสมกับธุรกิจขนาดเล็ก และขนาดกลางและไม่จำเป็น

ต้องสิ้นเปลืองงบประมาณจำนวนมากไปกับการซื้อระบบจัดการฐานข้อมูล แม้ว่า MySQL จะขาดคุณสมบัติบางอย่างเช่น Stored Procedures และ Triggers (ปัจจุบันสนับสนุน Transactions) แต่เพียงพอสำหรับระบบงานส่วนใหญ่ (Stored Procedures จะเริ่มมีใน MySQL เวอร์ชัน 5.0)

2.3 Web Application

Web Application ถือกำเนิดมาจากการที่เรามีเว็บไซต์ และ โดยพื้นฐานแล้ว Web Application ก็คือ การนำเว็บไซต์มาใส่การทำงานเพิ่มเติม เพื่อให้สามารถใช้แอปพลิเคชัน โดยมีเว็บเพจเป็นสื่อในการติดต่อระหว่างผู้ใช้งานกับระบบ จึงอาจสรุปได้ว่า การที่แอปพลิเคชันหนึ่ง ๆ จะเรียกได้ว่าเป็น Web Application นั้นต้องมีองค์ประกอบอย่างน้อย 3 อย่าง ได้แก่

1. Web Server ฟัง Client
2. Web Server
3. Application Server

Web Application เป็นระบบที่มีการทำงานในลักษณะของ Client-Server โดยมี Web Browser ทางฝั่งของไคลเอนต์เพื่อให้ไคลเอนต์สามารถติดต่อกับเว็บเซิร์ฟเวอร์ผ่านทางเว็บเพจ และใช้แอปพลิเคชันเซิร์ฟเวอร์ (Application Server) ในการจัดการส่วนของการทำงานหรือประมวลผล

ซึ่งเว็บเซิร์ฟเวอร์กับแอปพลิเคชันเซิร์ฟเวอร์ อาจจะอยู่บนเครื่องเดียวกัน หรืออยู่คนละเครื่องก็ได้
แล้วแต่ความเหมาะสมของระบบงาน

ส่วนรูปแบบเชิงสถาปัตยกรรมของ Web Application ที่มีการใช้งานในปัจจุบัน เราสามารถ
จำแนกออกเป็น 3 อย่างได้แก่

1. Thin Web Client ส่วนมากเป็นแอปพลิเคชันที่ทำงานบนเครือข่ายอินเทอร์เน็ต การทำงาน
ทั้งหมดจะทำงานอยู่ทางฝั่งของเซิร์ฟเวอร์มีการควบคุมการทำงานของไคลเอนต์เพียง
เล็กน้อย และไคลเอนต์จะใช้งานแอปพลิเคชันในรูปแบบฟอร์มเบราว์เซอร์
2. Thick Web Client เป็นแอปพลิเคชันที่มีส่วนของการทำงานอยู่บนฝั่งไคลเอนต์เป็นจำนวน
ค่อนข้างมาก โดยทั่วไปไคลเอนต์จะมีการใช้ Dynamic HTML, Java Applet หรือ ActiveX
ในการควบคุมการทำงานต่างๆ และมีการติดต่อกับเซิร์ฟเวอร์ฝ่ายโปรโตคอล HTTP
3. Web Delivery แบบนี้อาจจะมีการใช้โปรโตคอลอื่น ๆ ร่วมด้วย เช่น IIOP และ DCOM เพื่อ
ใช้สำหรับระบบแบบ Distributed Object โดยเว็บเบราว์เซอร์จะทำหน้าที่เป็นตัวเก็บและส่ง
Object ของระบบ

Thin Web Client

รูปแบบสถาปัตยกรรมแบบ Thin Web Client นั้น เป็นรูปแบบที่เหมาะสมสำหรับ Web
Application ส่วนใหญ่ ทั้งนี้เนื่องจากผู้ใช้สามารถเข้าใช้แอปพลิเคชันได้ด้วยโปรแกรมเว็บเบราว์เซอร์
ที่สนับสนุนโค้ด HTML มาตรฐาน ส่วนการประมวลผลต่างๆ นั้นจะเกิดขึ้นที่ฝั่งเซิร์ฟเวอร์ส่งผลให้
ผู้ดูแลระบบสามารถควบคุมการทำงานได้สะดวกกว่า

การประยุกต์ใช้งาน

สามารถนำไปประยุกต์ใช้กับ Web Application สำหรับ E-Commerce ส่วนใหญ่เป็น
รูปแบบสถาปัตยกรรมที่รองรับการใช้งานจากกลุ่มผู้ใช้งานขนาดใหญ่ ซึ่งมีความหลากหลายทั้ง
Platform ของเบราว์เซอร์ และการสนับสนุนของเบราว์เซอร์ทั้งนี้ในรูปแบบสถาปัตยกรรมนี้ เครื่อง
ไคลเอนต์จะทำหน้าที่รับโค้ด (Code) ภาษา HTML ไปแสดงผล และนำข้อมูลที่ได้กรอกลงใน
ฟอร์ม กลับไปยังเซิร์ฟเวอร์โดยใช้ CGI (Common Gateway Interface) ผ่านโปรโตคอล HTTP
เท่านั้น ซึ่งเบราว์เซอร์ทั้งหมดจะสนับสนุนคุณสมบัตินี้อยู่แล้ว

โครงสร้าง

ส่วนประกอบที่สำคัญในรูปแบบสถาปัตยกรรม Thin Web Client นั้นส่วนใหญ่ จะติดตั้ง และทำงานอยู่ในฝั่งของไคลเอนต์ทำหน้าที่ในการแสดงผล และรับข้อมูลจากผู้ใช้เท่านั้น สำหรับ ส่วนประกอบที่สำคัญนั้นก็คือ

- Browser ที่ไคลเอนต์ที่เป็น โปรแกรมมาตรฐานที่สนับสนุนคุณสมบัติฟอร์มของ HTML ทั่ว ๆ ไปเบราว์เซอร์นี้จะทำหน้าที่เป็นเหมือนกับอุปกรณ์กับติดต่อผู้ใช้พื้นฐาน เมื่อทีมพัฒนาเลือกรูปแบบสถาปัตยกรรมแบบ Thin Web Client ทีมพัฒนาไม่ควรจะ กำหนดให้ไคลเอนต์ประมวลผลมากกว่าการเก็บข้อมูลของผู้ใช้ด้วย Cookie ในมุมมอง ของผู้ใช้ เขาจะใช้เบราว์เซอร์ดึงเพจ HTML มาจาก Server ซึ่งในเพจนั้น ผู้ใช้จะกรอก ข้อมูลหรือข้อมูลที่ต้องการ จากนั้นก็จัดส่งข้อมูลกลับไปผ่าน CGI
- Web Server เป็นจุดหลักที่ใช้ในการเชื่อมต่อกลับเบราว์เซอร์ของผู้ใช้ไคลเอนต์ซึ่งเบราว์เซอร์ที่ไคลเอนต์ในสถาปัตยกรรมแบบนี้จะติดต่อกลับเว็บเซิร์ฟเวอร์เท่านั้น ในส่วน เว็บเซิร์ฟเวอร์นั้นจะทำหน้าที่รับการร้องขอเพจ HTML จากผู้ใช้ และอาจมีหน่วย ประมวลผลที่ได้รับการเขียนขึ้นด้วยภาษาสคริปต์ที่ฝั่ง Server งานอยู่ด้วย ถ้าผู้ใช้งาน ร้องขอเพจที่ต้องการประมวลผล ตัวเว็บเซิร์ฟเวอร์จะทำหน้าที่เรียกโมดูลของภาษา สคริปต์ที่ฝั่ง Server งานอยู่ด้วย ถ้าผู้ใช้งานร้องขอเพจที่ต้องการประมวลผล ตัวเว็บ เซิร์ฟเวอร์ จะทำหน้าที่เรียกโมดูลของภาษาสคริปต์ขึ้นมาทำงาน โดยอัตโนมัติ และได้ ผลลัพธ์เว็บเซิร์ฟเวอร์ก็จะทำหน้าที่สร้างเพจ HTML และส่งกลับไปให้กลับเบราว์เซอร์ ที่ไคลเอนต์
- การเชื่อมต่อ HTTP เป็นโปรโตคอลหลักที่ใช้ในการสื่อสารระหว่าง Brower ของไคล เอนต์กับ Web Server โดยการสื่อสารระหว่างไคลเอนต์กับ Server นั้นจะเป็นแบบ Connectionless ซึ่งหมายความว่า เมื่อไคลเอนต์หรือ Server จะสื่อสารกัน จำเป็นต้องมีการ เชื่อมต่อใหม่ทุกครั้งนอกจากการสื่อสารผ่าน HTTP โปรโตคอลแล้ว สำหรับ Web Application ที่ต้องการควบคุมความปลอดภัยของข้อมูลอาจจะมีการสื่อสารผ่าน โปรโตคอล SSL ซึ่งมีการเข้ารหัสก็ได้
- เพจ HTML เป็นเพจที่จะแสดงผลกับผู้ใช้ โดยจะเป็นเพจและข้อมูลสถิติที่ได้มีการ สร้างและเก็บไว้บน Server ก่อนแล้วในเพจ HTML นี้นอกจะเป็นส่วนที่ใช้ในการ แสดงผลข้อมูลแล้วยังอาจจะประกอบด้วยฟอร์ม (Form) สำหรับรับอินพุตจากผู้ใช้ เพื่อส่งกลับไปเรียก โมดูลภาษาสคริปต์ในฝั่ง Server เพื่อทำการประมวลผลด้วยก็ได้
- Server page เป็นเพจในการแสดงผลลัพธ์ของข้อมูล que ผู้ใช้ต้องการ ปกติแล้ว Server จะ เป็นส่วนที่เก็บและประมวลผลฝั่งสคริปต์ Server เพื่อตั้งให้ Web Server ประมวลผล และสร้าง โค้ด HTML สำหรับการแสดงผลต่อผู้ใช้ ซึ่งตัว Web Server อาจจะเรียกใช้

ส่วนประกอบหรือโมดูลที่อยู่ในแอปพลิเคชันเซิร์ฟเวอร์สำหรับการประมวลผลที่ซับซ้อนดังเช่นการทำทรานแซกชัน หรือเพื่อการติดต่อระบบฐานข้อมูลเป็นต้น

- Application Server เป็นส่วนหลักที่ทำหน้าที่ในการประมวลผล โดยปกติแล้ว จะเป็นส่วนที่เก็บและประมวลผลสคริปต์ที่ฝั่ง Server ซึ่งจะเรียกมาจาก Server Page อีกที่หนึ่งแอปพลิเคชันเซิร์ฟเวอร์นั้นอาจจะทำงานบนคอมพิวเตอร์เครื่องเดียวกันกับ Web Server ก็ได้ แต่วัตถุประสงค์หลักของแอปพลิเคชันเซิร์ฟเวอร์นั้นก็เพื่อให้มีการแบ่งส่วนที่ทำหน้าที่ในการแสดงผลออกจากส่วนสำหรับประมวลผลอย่างชัดเจน ซึ่งจะมีผลดีกับการเปลี่ยนแปลง และแก้ไข นอกจากนี้แอปพลิเคชันเซิร์ฟเวอร์ยังอาจจะติดต่อกับระบบฐานข้อมูล หรือระบบซอฟต์แวร์เดิมที่มีอยู่แล้วด้วยก็ได้ ในกรณีที่มีการเก็บข้อมูลมีความสำคัญ หรือต้องการนำเอาระบบที่มีอยู่แล้วกลับมาใช้งานด้วย

Thick Web Client

รูปแบบสถาปัตยกรรม Thick Web Client เป็นรูปแบบที่ได้รับการขยายเพิ่มขึ้นมาจากรูปแบบ Thin Web Client โดยในส่วนของไคลเอนต์นั้นนอกจากจะทำหน้าที่ในการแสดงผลและรับข้อมูลอินพุตมาจากผู้ใช้แล้ว ยังมีความสามารถในการประมวลผลบางอย่างด้วย ซึ่งในสถาปัตยกรรมนี้ Web Server จะไม่ได้เพียงแต่รับและแสดงผลโค้ด HTML เท่านั้น แต่ยังมีภาษาสคริปต์ฝั่งไคลเอนต์หรือออบเจกต์บางชนิด ดังเช่นตัวควบคุม ActiveX หรือจาวาแอปเพลต ทำงานอยู่ โดยชื่อของรูปแบบสถาปัตยกรรมนั้น ก็ตรงกับความหมายของมัน นั่นคือ ไคลเอนต์มีความสามารถในการประมวลผลทางธุรกิจพื้นฐานบางอย่างที่ไม่จำเป็นต้องติดต่อกับ Server ซึ่งต้องใช้เวลาและทรัพยากรมากกว่า และการใช้ภาษาสคริปต์หรือออบเจกต์นั้นยังช่วยให้ทีมพัฒนาสามารถเพิ่มส่วน ติดต่อกับผู้ใช้ ที่มีความซับซ้อนมากกว่าเข้าไปในแอปพลิเคชันได้ด้วย

การประยุกต์ใช้

ในรูปแบบสถาปัตยกรรมนี้จะใช้ภาษาสคริปต์ที่ฝั่งไคลเอนต์สำหรับเบราว์เซอร์ เพื่อการแสดงผลพิเศษตัวอย่างเช่น การใช้จาวาสคริปต์ในการสร้างภาพเคลื่อนไหว หรือใช้ในการตรวจสอบการครบถ้วนของข้อมูลที่ผู้ใช้กรอกเข้ามา หรืออาจจะใช้จาวาแอปเพลตหรือ ควบคุม ActiveX ในการแสดงข้อมูลในรูปแบบแผนภูมิต้นไม้ และนอกจากภาษาสคริปต์และออบเจกต์แล้ว และยังมีปลั๊กอินชนิดอื่นที่ใช้ในการนำเสนอข้อมูลที่มีความซับซ้อนได้ ดังเช่น Macromedia Shockwave ซึ่งมีความสามารถในการแสดงภาพเคลื่อนไหว หรือการคำนวณเกี่ยวกับกราฟิกและแอนิเมชันพื้นฐาน ซึ่งผู้พัฒนาสามารถเพิ่มความตื่นตาตื่นใจให้กับเว็บไซต์ด้วยภาพ และแอนิเมชันเคลื่อนไหว รวมทั้งยังมีความสามารถในการส่งเสียงอีกด้วย

โครงสร้าง

โครงสร้างส่วนใหญ่ของรูปแบบสถาปัตยกรรม Thick Web Client นั้นจะคล้ายกับ Thin Web Client พร้อมทั้งมีรูปแบบในการสื่อสารที่เหมือนกันด้วย นั่นคือ การสื่อสารระหว่างไคลเอนต์กับ Server นั้นจะทำงานผ่านโปรโตคอล HTTP โดยออบเจกต์ไม่ว่าจะเป็นภาษาจาวาแอปเพลตหรือตัวควบคุม ActiveX หรือภาษาสคริปต์ที่จะมาทำงานที่ไคลเอนต์ก็就会被ดาวน์โหลด (Download) มาผ่านโปรโตคอล HTTP เช่นกัน ส่วนการสื่อสารระหว่างไคลเอนต์กับ Server หลังจากนั้นอาจจะมีการใช้เอกสาร XML เพื่อกำหนดโครงสร้างของข้อมูลที่จะสื่อสารกัน เราอาจจะกล่าวได้ว่ารูปแบบสถาปัตยกรรม Thick Web Client โดยที่ตัวไคลเอนต์มีความสามารถในการประมวลผลด้วยออบเจกต์หรือภาษาสคริปต์ สำหรับส่วนประกอบที่สำคัญในรูปแบบสถาปัตยกรรม Thick Web Client นั้นประกอบด้วย

- Script Client ดังเช่น จาวาสคริปต์ หรือ VBScript ที่ฝังโค้ดอยู่ในเพจ HTML ตัวเบราว์เซอร์จะทำหน้าที่ตีความ และทำงานตามสคริปต์ที่ได้เขียนไว้
- ตัวควบคุม ActiveX เป็นออบเจกต์ COM ที่ได้รับการพัฒนา และนำเสนอโดยบริษัท Microsoft ตัวควบคุม ActiveX นั้นจะถูกดาวน์โหลดมาที่ไคลเอนต์และทำงานภายใต้สถานะแวดล้อมของเบราว์เซอร์แต่ถ้าจำเป็น ตัวควบคุม ActiveX ก็มีความสามารถในการติดต่อกับสิ่งที่อยู่นอกเบราว์เซอร์ เช่น การติดต่อกับฮาร์ดแวร์พื้นฐานบางประเภทได้เช่นกัน ซึ่งในกรณีนี้ ผู้ใช้จะต้องดำเนินการตรวจสอบและยอมรับให้ตัวควบคุม ActiveX นั้นสามารถทำงานที่มีความเสี่ยงได้
- จาวาแอปเพลต เป็นคอมโพเนนท์ที่ได้รับการคอมไพล์แล้ว ซึ่งจะถูกดาวน์โหลด และทำงานภายใต้สถานะแวดล้อมของ Browser จาวาแอปเพลตนั้น ไม่มีความสามารถในการทำงานนอกเหนือจากที่เบราว์เซอร์กำหนดไว้ ซึ่งทำให้มีความปลอดภัยมากกว่า นอกจากนั้น จาวาแอปเพลตยังสามารถใช้เป็นส่วนแสดงผลที่มีความซับซ้อนกว่าปกติได้ รวมทั้งยังมีความสามารถในการดึงข้อมูลจากเอกสาร

Web Delivery Client

สำหรับความหมายและการทำงานของรูปแบบสถาปัตยกรรม Web Delivery Client เหมือนกับชื่อที่บอกได้ว่า คือเว็บเป็นเพียงกลไกหลักในการส่งผลออบเจกต์แบบกระจายซึ่งทำงานแบบ Client/Server เราอาจกล่าวได้ว่ารูปแบบสถาปัตยกรรม Web Delivery Client นั้นเป็นสถาปัตยกรรมที่มีการทำงานเป็นออบเจกต์แบบกระจาย โดยออบเจกต์แบบกระจายซึ่งอยู่ที่ไคลเอนต์นั้นจะทำงานบนเบราว์เซอร์สำหรับการถ่ายทอดโดยออบเจกต์เพื่อมาทำงานที่ไคลเอนต์นั้นทำ

ผ่าน โปรโตคอล HTTP แต่การสื่อสารหลังจากนั้นจะทำผ่านโปรโตคอลของออบเจกต์แบบกระจายแทน รูปแบบสถาปัตยกรรม Web Delivery Client ซึ่งการโมเดลและการออกแบบที่ใช้รูปแบบสถาปัตยกรรมนี้จึงมีรายละเอียดมากกว่ารูปแบบสถาปัตยกรรมอื่น ซึ่งทีมพัฒนาอาจจะจำเป็นต้องเลือกใช้รูปแบบการออกแบบอื่นที่เหมาะสมประกอบกันด้วย

การประยุกต์ใช้

รูปแบบสถาปัตยกรรม Web Delivery Client นี้เหมาะสำหรับแอปพลิเคชันหรือบางส่วนของแอปพลิเคชันที่ต้องการเชื่อมต่อซึ่งมีความเชื่อมต่อได้สูง รวมทั้งจะเหมาะกับแอปพลิเคชันที่ไคลเอนต์สามารถแบ่งงานหรือการควบคุมบางอย่างจาก Server มาทำได้ ทั้งนี้เนื่องจากการสื่อสารระหว่างไคลเอนต์กับ Server ในสถาปัตยกรรมนี้จะทำผ่าน โปรโตคอลสำหรับการทำงานของออบเจกต์แบบกระจายซึ่งเกือบทั้งหมดเป็นแบบ Connection-Oriented นอกจากนี้ในการทำงานแบบออบเจกต์แบบกระจายนั้นไคลเอนต์กับ Server จะมีความปลอดภัยที่สูงกว่าการเชื่อมต่อผ่าน โปรโตคอล HTTP ปกติ ดังนั้น งานซึ่งมีความสำคัญที่ต้องการควบคุมจากไคลเอนต์จึงเหมาะสมสำหรับรูปแบบสถาปัตยกรรมนี้

โครงสร้าง

สำหรับส่วนที่รูปแบบสถาปัตยกรรม Web Delivery Client แตกต่างจากรูปแบบสถาปัตยกรรมของ Web Application ปกตินั้น อยู่ที่ส่วนสำหรับการสื่อสารระหว่างไคลเอนต์กับ Server โดยในรูปแบบสถาปัตยกรรมอื่นนั้นการสื่อสารทั้งหมดจะเกิดขึ้นผ่านโปรโตคอล HTTP ซึ่งเป็นการสื่อสารแบบ Connectionless ที่มีข้อจำกัดสำหรับแอปพลิเคชันที่ต้องการมีการโต้ตอบแบบทันทีทันใดระหว่างผู้ใช้กับ Server อย่างไรก็ตามส่วนประกอบที่สำคัญของรูปแบบสถาปัตยกรรม Web Delivery Client นั้นยังคงคล้ายกับรูปแบบสถาปัตยกรรม Thin Web Client แต่มีส่วนที่เพิ่มและแตกต่างกันดังนี้ คือ

- DCOM หรือเทคโนโลยี Distributed COM สำหรับการประมวลผลของออบเจกต์แบบกระจายที่ได้รับการพัฒนา และนำเสนอโดยบริษัท Microsoft
- ActiveX Server Component หรือ COM+ เป็นส่วนที่ทำหน้าที่ในการรองรับการเชื่อมต่อจากตัวควบคุม ActiveX ที่ไคลเอนต์โดยสื่อสารผ่านโมเดล DCOM
- IIOP (Internet Inter – ORB Protocol) ซึ่งเป็นโปรโตคอลสำหรับเทคโนโลยีออบเจกต์แบบกระจาย CORBA ที่ได้รับการกำหนดขึ้นโดย OMG เป็นโปรโตคอลในการสื่อสารระหว่างออบเจกต์แบบกระจายที่สามารถทำงานผ่านโปรโตคอล TCP/IP ของอินเทอร์เน็ตปกติได้

- RMI (Java Remote Method Protocol , JRMP) หรือ Remote Method Invocation เป็นเทคโนโลยีของออบเจกต์แบบกระจายสำหรับจาวา โดยในปัจจุบันโปรโตคอล RMI นั้นสามารถทำได้บน IIOP ของ CORBA
- Enterprise Java (EJB) เป็นส่วนประกอบที่ทำหน้าที่รองรับจากการเชื่อมต่อจาวาแอปเพลต

โดยสามารถสื่อสารผ่าน RMI เอง หรือ RMI ที่ทำงานอยู่บนโมเดล IIOP ของ CORBA ก็ได้

จากโครงสร้างจะเห็นว่าออบเจกต์ที่ทำงานแบบกระจายนั้น จะแบ่งออก 2 ค่าย ใหญ่ ๆ ก็คือ โมเดลที่ได้รับการคิดค้น และนำเสนอโดยบริษัท Microsoft ซึ่งเป็นออบเจกต์ที่มีพื้นฐานการทำงานบนเทคโนโลยีระบบปฏิบัติการวินโดวส์ และ Internet Explorer ของบริษัท Microsoft เอง ส่วนอีกเทคโนโลยีหนึ่ง คือเทคโนโลยีของจาวาที่ได้รับการคิดค้น และนำเสนอโดยบริษัท Sun สองโมเดลนี้ไม่สามารถทำงานร่วมกันได้โดยตรง ซึ่งการที่ผู้พัฒนาจะเลือกเทคโนโลยีใดนั้น ก็แล้วแต่ความสะดวก และความเหมาะสมของแอปพลิเคชัน แต่ถ้าจำเป็นต้องเลือกใช้ทั้งสองเทคโนโลยี ผู้พัฒนาก็มีทางเลือกที่สามารถจะสื่อสารระหว่างออบเจกต์แบบกระจายได้โดยการใช้เทคโนโลยี SOAP (Simple Object Access Protocol)

SOAP เป็นเทคโนโลยีมาตรฐานกลางที่ใช้ในการสื่อสารระหว่างออบเจกต์แบบกระจายที่ได้รับการพัฒนา และนำเสนอขึ้นโดยบริษัทต่าง ๆ โปรโตคอล SOAP มีพื้นฐานการทำงานโดยแลกเปลี่ยนข้อมูลผ่านเอกสาร XML โดยในเอกสาร XML ของ โปรโตคอล SOAP จะแบ่งออกเป็น 3 ส่วนก็คือ ส่วนที่ทำหน้าที่อธิบายเมสเสจ และวิธีการประมวลผล ส่วนที่ทำหน้าที่อธิบายกฎในการเข้ารหัสของข้อมูลสำหรับแอปพลิเคชันหรือออบเจกต์แบบกระจายแต่ละชนิด และส่วนที่ทำหน้าที่บ่งบอกวิธีการเรียกใช้งานออบเจกต์จากระยะไกล รวมทั้งการตอบสนอง โปรโตคอล SOAP นี้สามารถเชื่อมต่อออบเจกต์แบบกระจายได้โดยผ่าน โปรโตคอล HTTP